

Rekurze II, backtracking

IB111 ZÁKLADY PROGRAMOVÁNÍ

Tomáš Vojnar (na základě slajdů Nikoly Beneše)

21. listopadu 2025

Chceme získat řešení ve formě seznamu dvojic přesunů.

Chceme získat řešení ve formě seznamu dvojic přesunů.

```
Moves = list[tuple[str, str]]
```

```
def hanoi(num: int, start: str,
          end: str, aux: str) -> Moves:
    if num == 0:
        return []
    return (
        hanoi(num - 1, start, aux, end) +
        [(start, end)] +
        hanoi(num - 1, aux, end, start)
    )
```

Chceme získat řešení ve formě seznamu dvojic přesunů.

```
Moves = list[tuple[str, str]]
```

```
def hanoi(num: int, start: str,
          end: str, aux: str) -> Moves:
    if num == 0:
        return []
    return (
        hanoi(num - 1, start, aux, end) +
        [(start, end)] +
        hanoi(num - 1, aux, end, start)
    )
```

- Opakovaně vytváříme spoustu nových seznamů.
 - Jejich obsah často kopírujeme.

Chceme získat řešení ve formě seznamu dvojic přesunů.

```
Moves = list[tuple[str, str]]
```

```
def hanoi(num: int, start: str,
          end: str, aux: str) -> Moves:
    if num == 0:
        return []
    return (
        hanoi(num - 1, start, aux, end) +
        [(start, end)] +
        hanoi(num - 1, aux, end, start)
    )
```

- Opakovaně vytváříme spoustu nových seznamů.
 - Jejich obsah často kopírujeme.
 - Jak to udělat lépe?

```
def hanoi_better(num: int, start: str,
                 end: str, aux: str) -> Moves:
    return hanoi_aux(num, start, end, aux, [])

def hanoi_aux(num: int, start: str, end: str,
              aux: str, moves: Moves) -> Moves:
    if num > 0:
        hanoi_aux(num - 1, start, aux, end, moves)
        moves.append((start, end))
        hanoi_aux(num - 1, aux, end, start, moves)
    return moves
```

```
def hanoi_better(num: int, start: str,
                 end: str, aux: str) -> Moves:
    return hanoi_aux(num, start, end, aux, [])
```

Všimněte si

- Hlavní funkce `hanoi_better` je *čistá*.
- Pomocná funkce `hanoi_aux` není čistá:
 - modifikuje parametr `moves`, ale nemá jiné vedlejší efekty.
 - Někdy se takové funkce nazývají *polo-čisté* (semi-pure).
 - Neporušuje čistotu funkce `hanoi_better`, protože parametr `moves` vznikl uvnitř `hanoi_better`.

Vnořený seznam čísel

Vnořený seznam čísel je seznam, který obsahuje čísla nebo vnořené seznamy čísel.

```
nested_list = [[1, 2], 3, [4, 5, 6], [[[7, 8], 9], 10]]
```

- Rekurzivní datová struktura.

Vnořený seznam čísel je seznam, který obsahuje čísla nebo vnořené seznamy čísel.

```
nested_list = [[1, 2], 3, [4, 5, 6], [[[7, 8], 9], 10]]
```

- Rekurzivní datová struktura.
- Jak typově anotovat?
 - Současná verze mypy podporuje *rekurzivní typy*
 - (v dřívějších verzích bylo potřeba vhodně použít třídy).

Vnořený seznam čísel je seznam, který obsahuje čísla nebo vnořené seznamy čísel.

```
nested_list = [[1, 2], 3, [4, 5, 6], [[[7, 8], 9], 10]]
```

- Rekurzivní datová struktura.
- Jak typově anotovat?
 - Současná verze mypy podporuje *rekurzivní typy*
 - (v dřívějších verzích bylo potřeba vhodně použít třídy).
- Potřebujeme *typové sjednocení* |.
- Potřebujeme *dopřednou referenci*.
 - Připomenutí: dopřednou referenci s | je třeba napsat do jednoho řetězce.

```
NestedListItem = 'int | NestedList'  
NestedList = list[NestedListItem]
```

- Tohle nebude fungovat:
 - **typový alias nesmí celý být jen řetězec**
 - (nový způsob typových aliasů od Pythonu 3.12 tohle řeší, ale my zatím zůstáváme u 3.10).

```
NestedListItem = 'int | NestedList'  
NestedList = list[NestedListItem]
```

- Tohle už fungovat bude:

```
NestedList = list['NestedListItem']  
NestedListItem = int | NestedList
```

- Tohle nebude fungovat:
 - **typový alias nesmí celý být jen řetězec**
 - (nový způsob typových aliasů od Pythonu 3.12 tohle řeší, ale my zatím zůstáváme u 3.10).

```
NestedListItem = 'int | NestedList'  
NestedList = list[NestedListItem]
```

- Tohle už fungovat bude:

```
NestedList = list['NestedListItem']  
NestedListItem = int | NestedList
```

- Ale stačí také:

```
NestedList = list['int | NestedList']
```



```
def nested_list_sum1(nl: NestedList) -> int:
    total = 0
    for elem in nl:
        if isinstance(elem, list):
            total += nested_list_sum1(elem)
        else:
            # mypy knows the type of elem is int here
            total += elem

    return total
```

```
def nested_list_sum1(nl: NestedList) -> int:
    total = 0
    for elem in nl:
        if isinstance(elem, list):
            total += nested_list_sum1(elem)
        else:
            # mypy knows the type of elem is int here
            total += elem

    return total
```

- Šlo by to vylepšit?


```
def nested_list_sum2(value: int | NestedList) -> int:
    if isinstance(value, int):
        return value

    # mypy knows the type of value is NestedList here
    total = 0
    for item in value:
        total += nested_list_sum2(item)

    return total
```

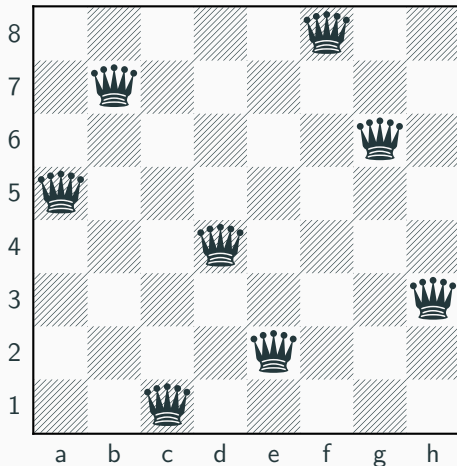
- Rozvolnění/zeslabení vstupní podmínky rekurzivní funkce.
 - (Podobný trik znáte možná pod názvem „zesílení indukce“.)

Backtracking

Hrr na ně, a když to nepude, tož cónem.

Problém dam (*N Queens Problem*)

- Jak umístit N dam na šachovnici o rozměrech $N \times N$ tak, aby se vzájemně neohrožovaly?



- Příklad *problému splnění omezení* (constraint satisfaction problem).

Naivní řešení (řešení hrubou silou)

- Vyzkoušíme všechna možná rozmístění N dam na šachovnici.
- Pokud najdeme takové, kde se dámy vzájemně neohrožují, skončíme.

- Příklad *problému splnění omezení* (constraint satisfaction problem).

Naivní řešení (řešení hrubou silou)

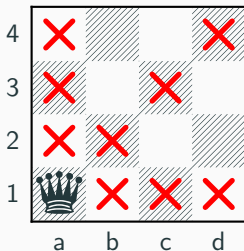
- Vyzkoušíme všechna možná rozmístění N dam na šachovnici.
- Pokud najdeme takové, kde se dámy vzájemně neohrožují, skončíme.
- Všech možných rozmístění 8 dam na klasické šachovnici je

$$\binom{64}{8} = 4\,426\,165\,368.$$

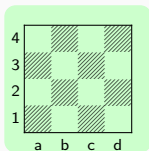
- I kdybychom jedno rozmístění zvládli zkontrolovat za 1 ms, řešili bychom tento problém více než 50 dní.

Lepší řešení

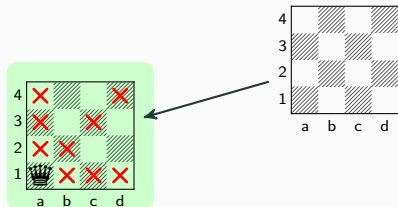
- Budujeme si situaci na šachovnici *postupně*.
- Novou dámu na šachovnici přidáme, pokud se vzájemně neohrožuje s žádnými už umístěnými dámami.
- Pokud už nemáme žádnou možnost, jak pokračovat, vrátíme se o něco zpět a zkusíme jinou cestu: *backtracking*.



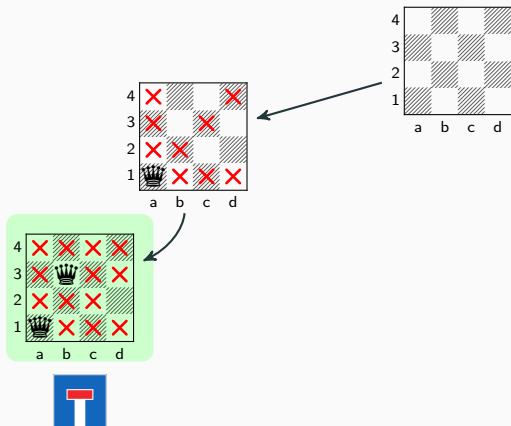
Problém dam – rekurzivní řešení



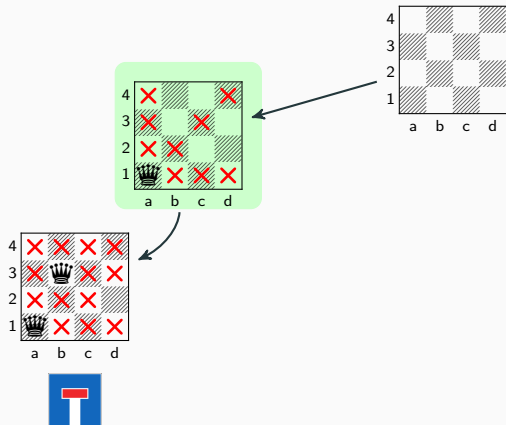
Problém dam – rekurzivní řešení



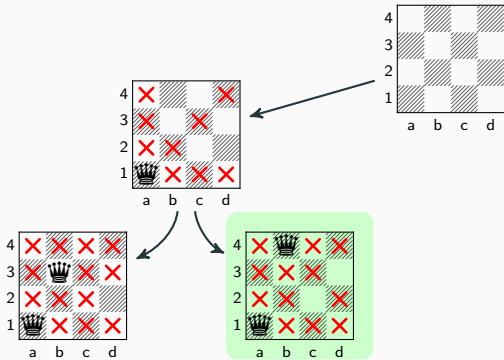
Problém dam – rekurzivní řešení



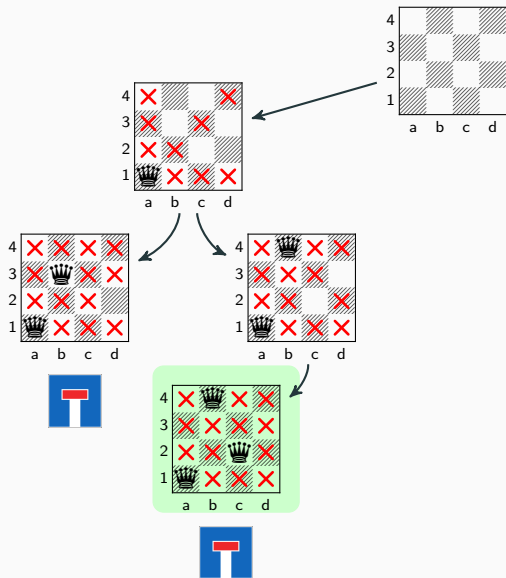
Problém dam – rekurzivní řešení



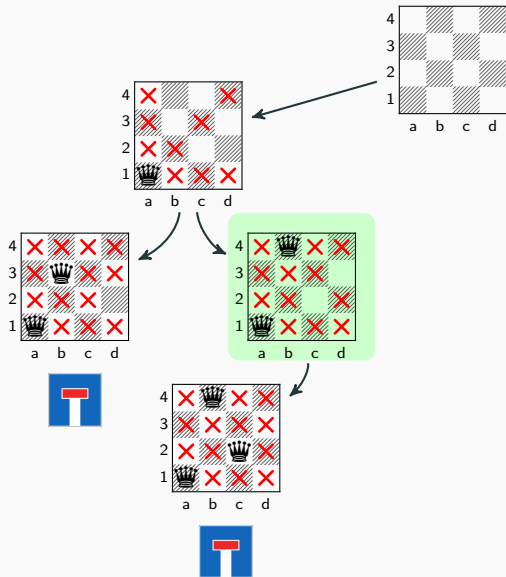
Problém dam – rekurzivní řešení



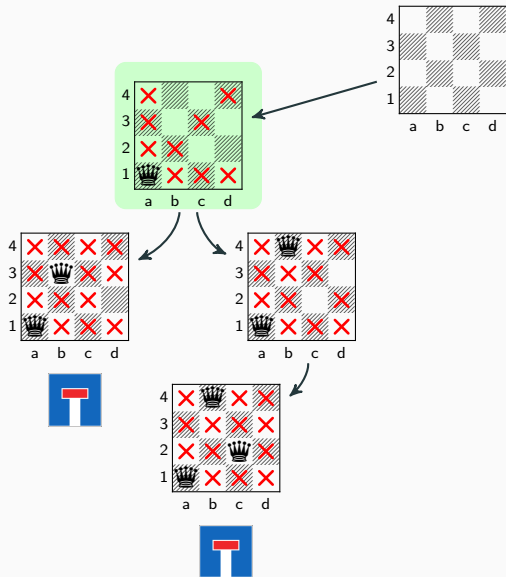
Problém dam – rekurzivní řešení



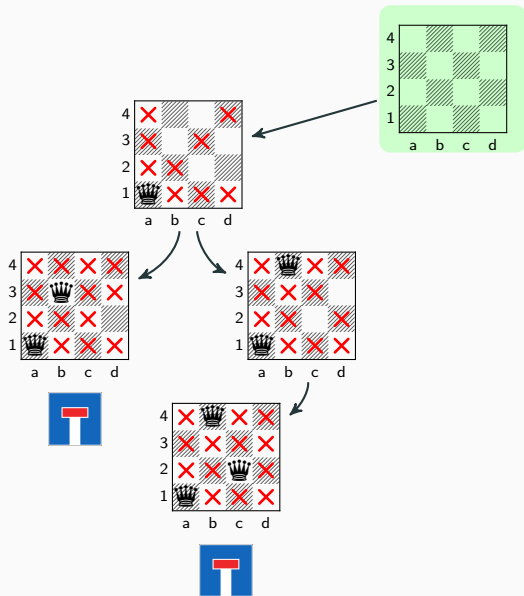
Problém dam – rekurzivní řešení



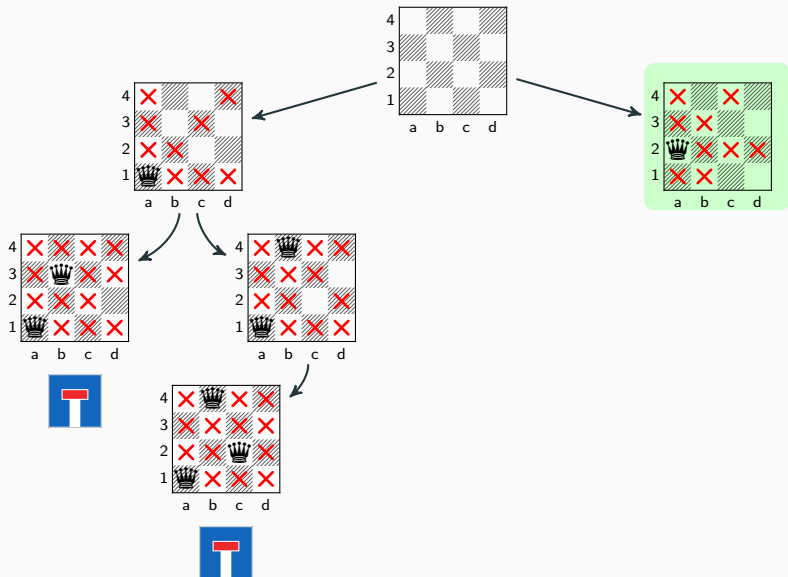
Problém dam – rekurzivní řešení



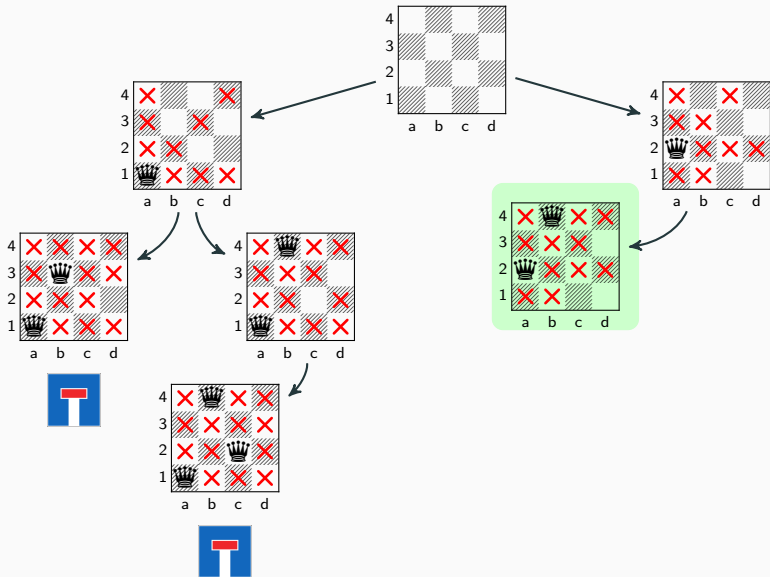
Problém dam – rekurzivní řešení



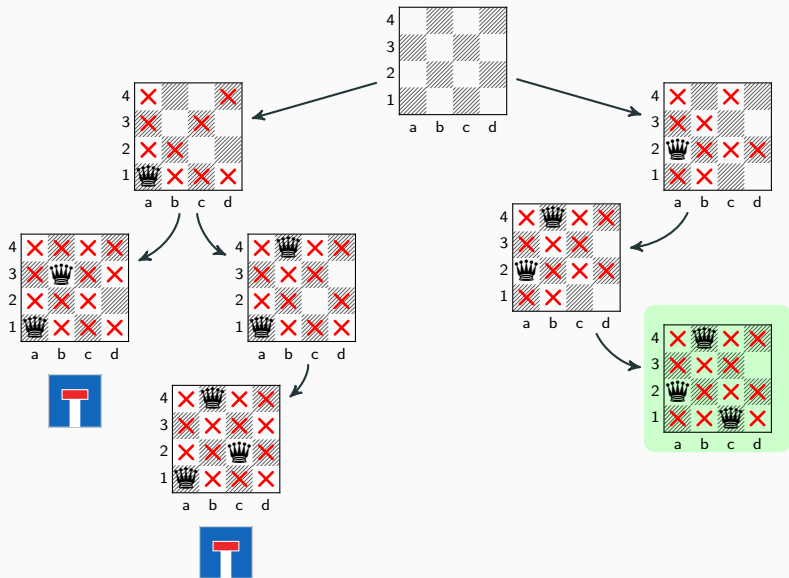
Problém dam – rekurzivní řešení



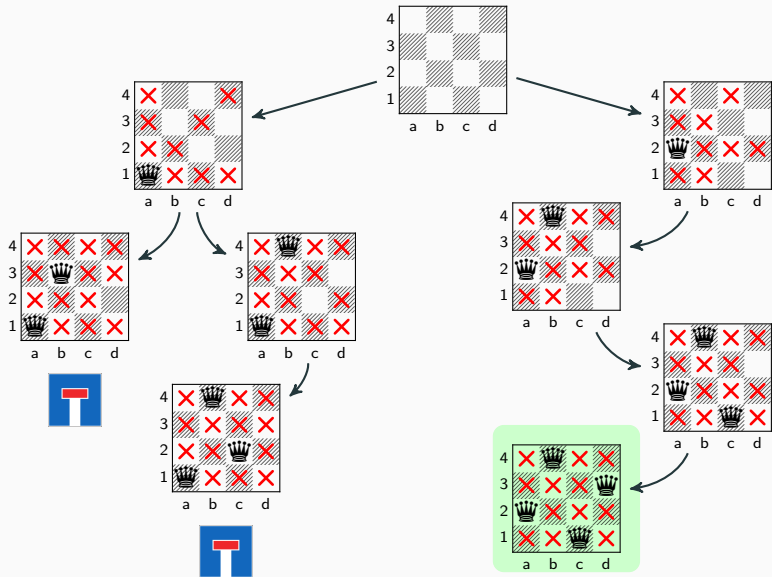
Problém dam – rekurzivní řešení



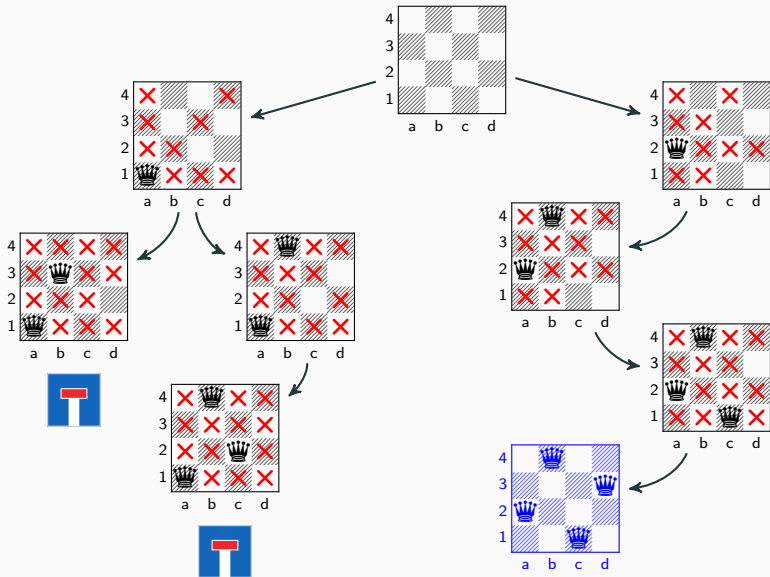
Problém dam – rekurzivní řešení



Problém dam – rekurzivní řešení



Problém dam – rekurzivní řešení



Důležitá pozorování

- V každém sloupci a každém řádku musí stát právě jedna dáma.
- Můžeme tedy situaci budovat např. po sloupcích.
- Jak reprezentovat šachovnici?

Důležitá pozorování

- V každém sloupci a každém řádku musí stát právě jedna dáma.
- Můžeme tedy situaci budovat např. po sloupcích.
- Jak reprezentovat šachovnici?
 - 2D matice.
 - Seznam pozic všech dam.
 - Pro každý sloupec si pamatovat řádek, na kterém je dáma.

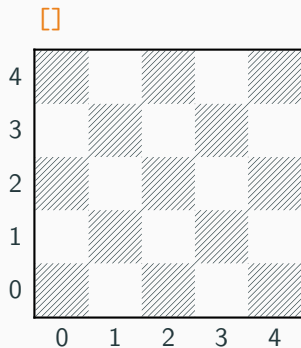
Důležitá pozorování

- V každém sloupci a každém řádku musí stát právě jedna dáma.
- Můžeme tedy situaci budovat např. po sloupcích.
- Jak reprezentovat šachovnici?
 - 2D matice.
 - Seznam pozic všech dam.
 - Pro každý sloupec si pamatovat řádek, na kterém je dáma.
- Jak poznat, že nově přidávaná dáma někoho ohrožuje?

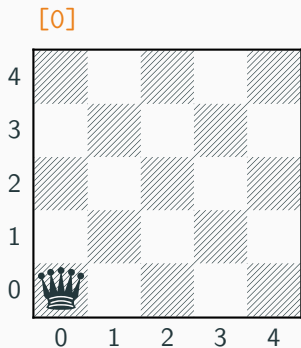
Důležitá pozorování

- V každém sloupci a každém řádku musí stát právě jedna dáma.
- Můžeme tedy situaci budovat např. po sloupcích.
- Jak reprezentovat šachovnici?
 - 2D matice.
 - Seznam pozic všech dam.
 - Pro každý sloupec si pamatovat řádek, na kterém je dáma.
- Jak poznat, že nově přidávaná dáma někoho ohrožuje?
 - Číslo řádku už je použito (kontrola řádků).
 - Jak poznat diagonály?

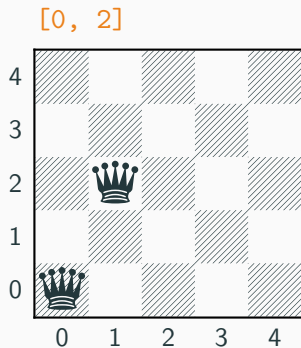
Reprezentace šachovnice



Reprezentace šachovnice

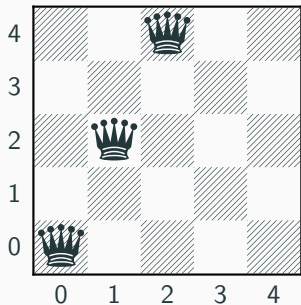


Reprezentace šachovnice



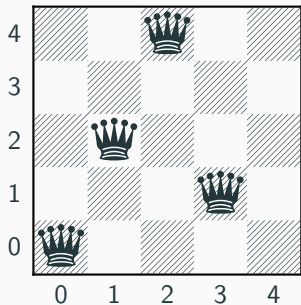
Reprezentace šachovnice

[0, 2, 4]



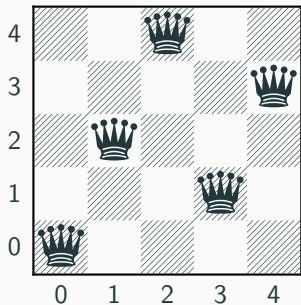
Reprezentace šachovnice

[0, 2, 4, 1]

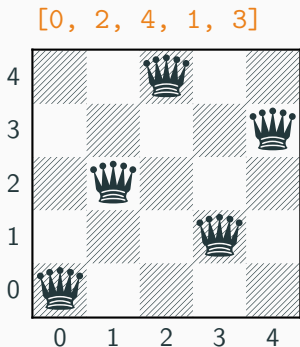


Reprezentace šachovnice

[0, 2, 4, 1, 3]



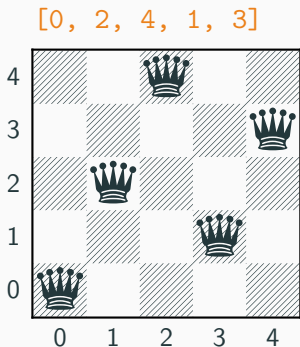
Reprezentace šachovnice



Diagonály

Kdy jsou dámy se souřadnicemi (x_1, y_1) a (x_2, y_2) na stejné diagonále?

Reprezentace šachovnice



Diagonály

Kdy jsou dámy se souřadnicemi (x_1, y_1) a (x_2, y_2) na stejné diagonále? Právě tehdy, když $|x_1 - x_2| = |y_1 - y_2|$.


```
def queen_check(state: list[int], new_row: int) -> bool:
    new_col = len(state)
    for col, row in enumerate(state):
        if row == new_row or \
            abs(row - new_row) == abs(col - new_col):
            return False
    return True
```

- `state` je seznam už umístěných dam.
- `new_row` je řádek, kam chceme nově umístit dámu.

```
def queens_recursive(state: list[int], size: int) \
    -> list[int] | None:
    if len(state) == size:
        # solution!
        return state

    for row in range(size):
        if queen_check(state, row):
            state.append(row)
            result = queens_recursive(state, size)
            if result is not None:
                return result
            state.pop()

    return None
```

```
def queens(size: int) -> list[int] | None:  
    return queens_recursive([], size)
```

Všimněte si

- Podobný princip jako u Hanojských věží dříve.
 - Hlavní funkce (čistá): nová datová struktura pro výsledek.
 - Pomocná rekurzivní funkce: modifikace struktury, přeposílání (*dolů*: rekurzivní volání, *nahoru*: návratová hodnota).
- Zde navíc existuje možnost, že rekurzivní volání *neuspěje*.
 - Pak nutno vrátit datovou strukturu do původního stavu.

Chceme *všechna* řešení

- Jedna možnost:
 - Vracet z pomocné funkce seznam řešení.
 - Při rekurzi řetězíme seznamy.
- Druhá možnost:
 - Opět použít předchozí myšlenku.
 - Přidat další parametr pro seznam řešení,
 - nalezená řešení vkládat do seznamu.

```
def queens(size: int) -> list[list[int]]:
    return queens_recursive([], [], size)

def queens_recursive(state: list[int],
                      solutions: list[list[int]],
                      size: int) -> list[list[int]]:
    if len(state) == size:
        solutions.append(state.copy())
        return solutions
    for row in range(size):
        if queen_check(state, row):
            state.append(row)
            queens_recursive(state, solutions, size)
            state.pop()
    return solutions
```

```
def queens(size: int) -> list[list[int]]:
    return queens_recursive([], [], size)

def queens_recursive(state: list[int],
                      solutions: list[list[int]],
                      size: int) -> list[list[int]]:
    if len(state) == size:
        solutions.append(state.copy())
        return solutions
    for row in range(size):
        if queen_check(state, row):
            state.append(row)
            queens_recursive(state, solutions, size)
            state.pop()
    return solutions
```

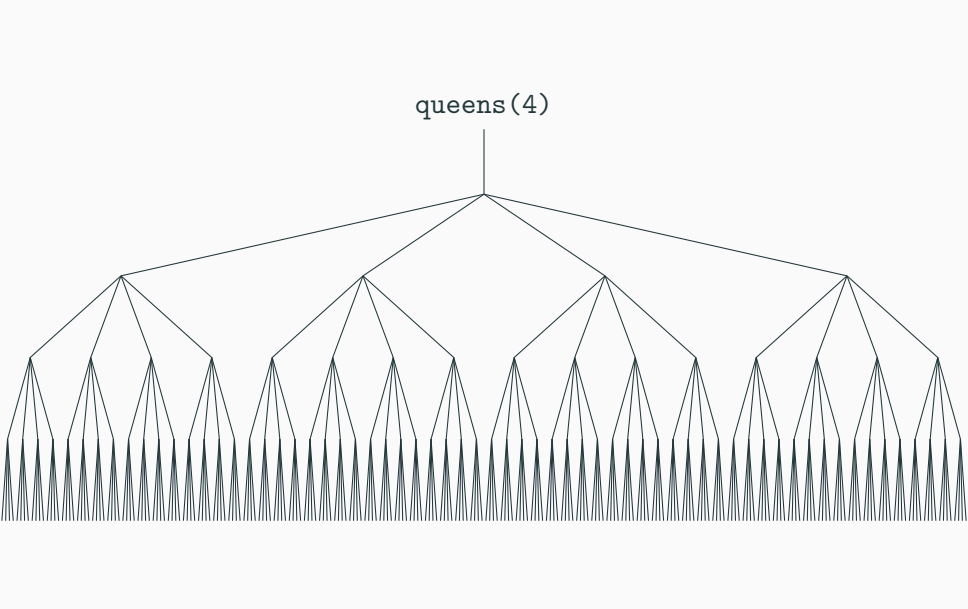
Co kdyby zde nebylo `.copy()`?
Zkuste použít pythontutor.com

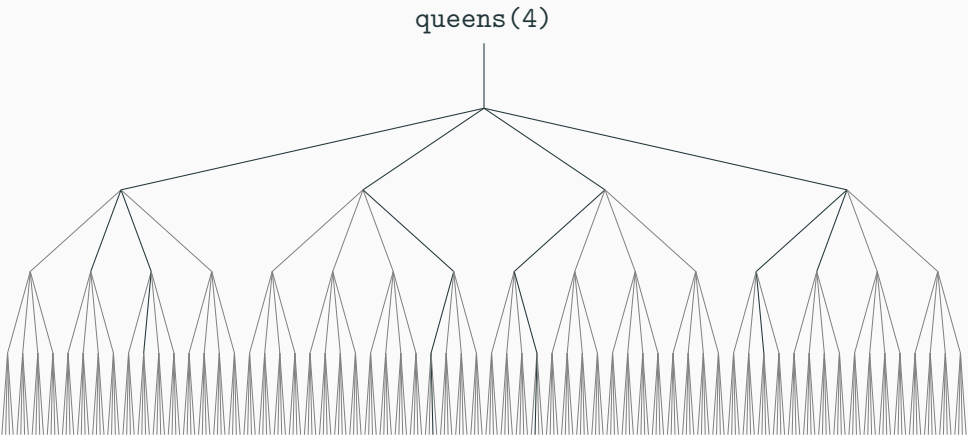
- Obecná technika pro řešení (mj.) problémů splnění omezení.
 - Nemusí být nejefektivnější.
 - Pořád neinformované hledání.
- Typicky s pomocí rekurze.
- Včas „zařízíme“ ty cesty výpočtu, které určitě nevedou k cíli.

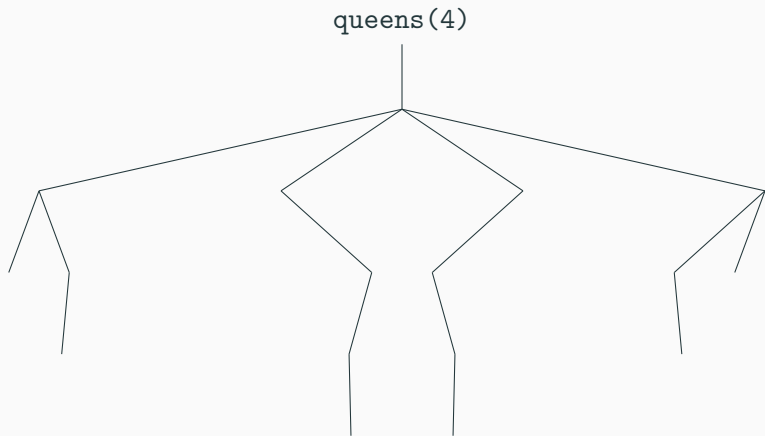
Problémy, které se dají řešit podobným způsobem:

- různé logické hádanky (např. sudoku),
- optimální strategie ve hrách dvou hráčů,
- optimalizační problémy (plánování, rozvrhování),
- ...

Strom rekurze







Varianty problému (pro domácí procvičení)

- Některé pozice už jsou předem obsazené dámami,
 - s předem umístěnými dámami nesmíme hýbat:
 - „N-Queens *Completion* Problem“.
 - Lehčí varianta: dámy jsou umístěny „zleva“.
- Některé pozice jsou zakázané:
 - nesmíme na ně umístit dámu.

- **Vstup:** kladné celé číslo n (požadovaný ciferný součet).
- **Výstup:** seznam všech čísel, která
 - mají v desítkovém zápise požadovaný ciferný součet n ,
 - neobsahují v desítkovém zápise žádnou nulu

- **Vstup:** kladné celé číslo n (požadovaný ciferný součet).
- **Výstup:** seznam všech čísel, která
 - mají v desítkovém zápise požadovaný ciferný součet n ,
 - neobsahují v desítkovém zápise žádnou nulu (aby jich bylo jen konečně mnoho).

Naivní řešení

- Postupně procházet všechna čísla od 1 do 111...111 (n krát) a kontrolovat jejich ciferné součty.
- Zkuste si implementovat a zjistěte, pro jaké vstupy už toto řešení trvá příliš dlouho.

Řešení pomocí backtrackingu

Řešení pomocí backtrackingu

```
def digit_sum_numbers(total: int) -> list[int]:  
    return dsn_rec(total, [], 0)  
  
def dsn_rec(remaining: int, result: list[int],  
            current: int) -> list[int]:  
    if remaining == 0:  
        result.append(current)  
    elif remaining > 0:  
        for digit in range(1, 10):  
            dsn_rec(remaining - digit, result,  
                    current * 10 + digit)  
    # else: nothing -- solution missed  
  
    return result
```

Příklad: Placení mincemi

- **Vstup:** částka, kterou chceme zaplatit; seznam hodnot mincí.
- **Výstup:** minimální počet mincí, které musíme použít k zaplacení částky (nebo informace o tom, že částku zaplatit nelze).

- **Vstup:** částka, kterou chceme zaplatit; seznam hodnot mincí.
- **Výstup:** minimální počet mincí, které musíme použít k zaplacení částky (nebo informace o tom, že částku zaplatit nelze).
- Pro běžné sady typu 1, 2, 5, 10, 20, 50 jde o jednoduchý problém,
- ale co třeba sada mincí 1, 10, 25?

- **Vstup:** částka, kterou chceme zaplatit; seznam hodnot mincí.
- **Výstup:** minimální počet mincí, které musíme použít k zaplacení částky (nebo informace o tom, že částku zaplatit nelze).
- Pro běžné sady typu 1, 2, 5, 10, 20, 50 jde o jednoduchý problém,
- ale co třeba sada mincí 1, 10, 25?
- Ukážeme si rekurzivní řešení pomocí backtrackingu.
- (Existuje i lepší řešení s využitím pokročilých technik návrhu algoritmů.)

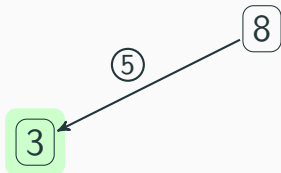
Placení mincemi

hodnoty mincí: 1, 4, 5; částka k zaplacení: 8



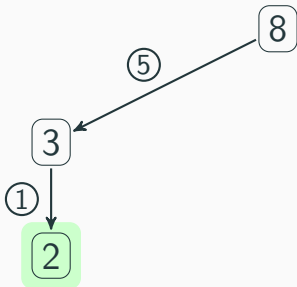
Placení mincemi

hodnoty mincí: 1, 4, 5; částka k zaplacení: 8



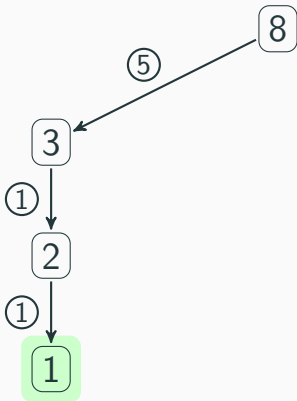
Placení mincemi

hodnoty mincí: 1, 4, 5; částka k zaplacení: 8



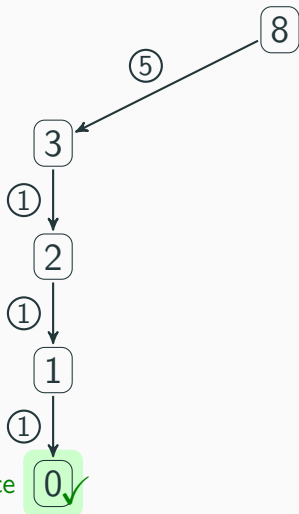
Placení mincemi

hodnoty mincí: 1, 4, 5; částka k zaplacení: 8



Placení mincemi

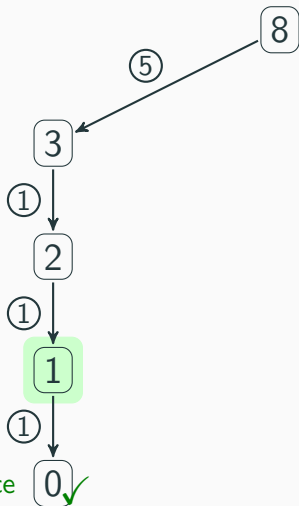
hodnoty mincí: 1, 4, 5; částka k zaplacení: 8



řešení: 4 mince

Placení mincemi

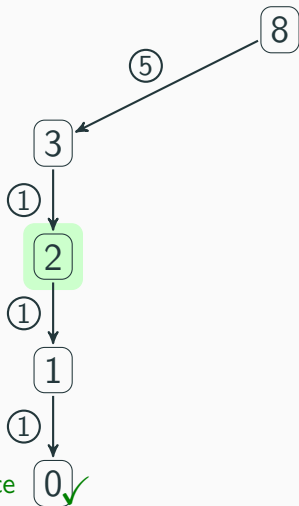
hodnoty mincí: 1, 4, 5; částka k zaplacení: 8



řešení: 4 mince

Placení mincemi

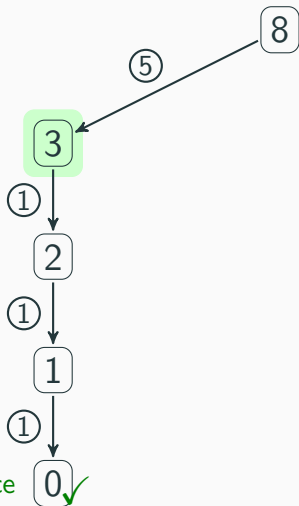
hodnoty mincí: 1, 4, 5; částka k zaplacení: 8



řešení: 4 mince

Placení mincemi

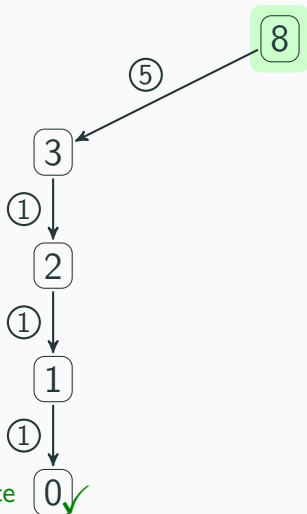
hodnoty mincí: 1, 4, 5; částka k zaplacení: 8



řešení: 4 mince

Placen1 mincem1

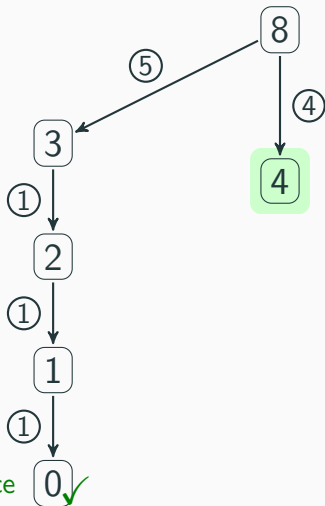
hodnoty minc1: 1, 4, 5; 1stka k zaplacen1: 8



řešení: 4 mince

Placení mincemi

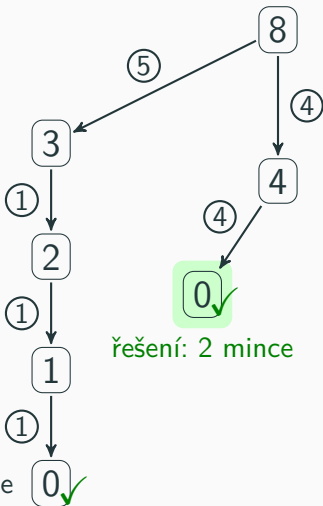
hodnoty mincí: 1, 4, 5; částka k zaplacení: 8



řešení: 4 mince

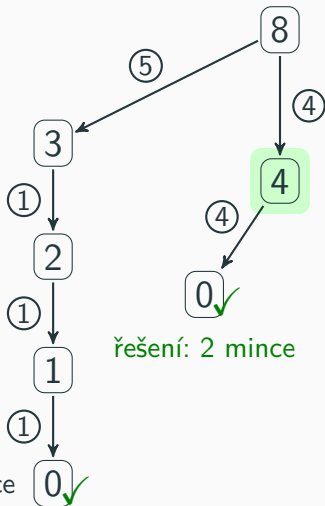
Placení mincemi

hodnoty mincí: 1, 4, 5; částka k zaplacení: 8



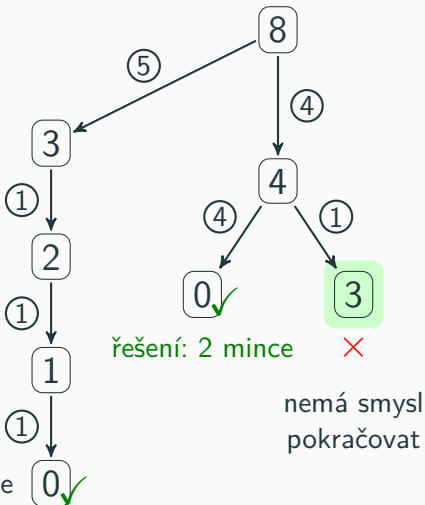
Placení mincemi

hodnoty mincí: 1, 4, 5; částka k zaplacení: 8



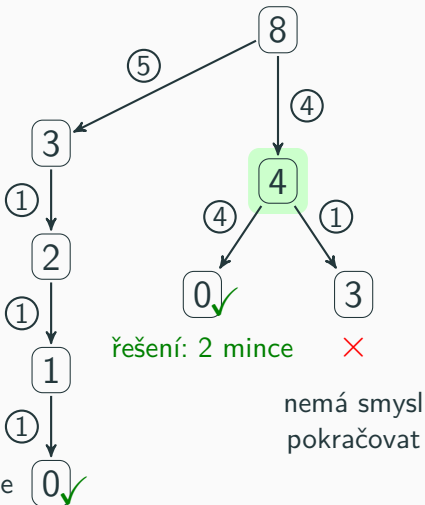
Placení mincemi

hodnoty mincí: 1, 4, 5; částka k zaplacení: 8



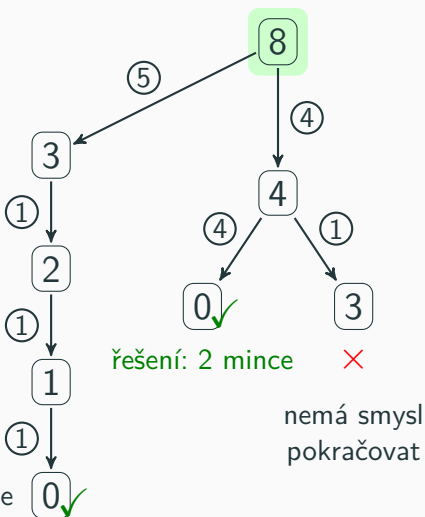
Placení mincemi

hodnoty mincí: 1, 4, 5; částka k zaplacení: 8



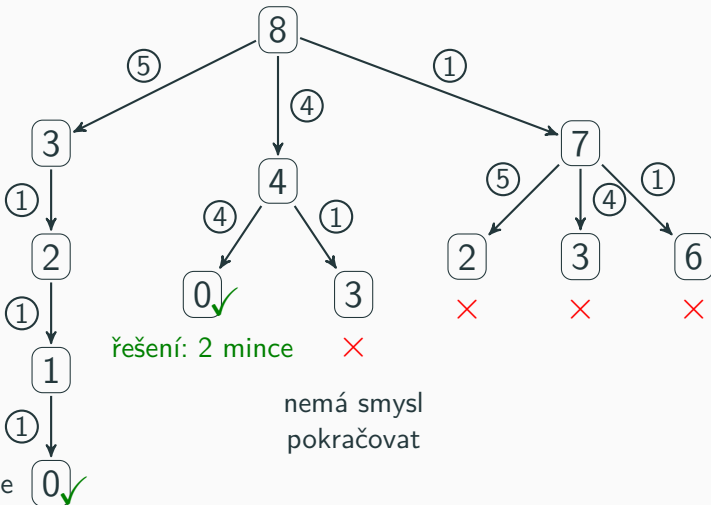
Placení mincemi

hodnoty mincí: 1, 4, 5; částka k zaplacení: 8



Placení mincemi

hodnoty mincí: 1, 4, 5; částka k zaplacení: 8



```
def min_coins(amount: int, coins: list[int]) \
    -> int | None:
    return min_coins_rec(amount, coins, 0, None)

def min_coins_rec(amount: int, coins: list[int],
                  count: int, best: int | None) \
    -> int | None:
    if count == best or amount == 0:
        return count

    for coin in coins:
        if amount >= coin:
            best = min_coins_rec(amount - coin, coins,
                                count + 1, best)

    return best
```

Náměty k rozšíření (pro domácí procvičení)

- Vylepšit rekurzi, aby se zastavila o něco dříve.
- Místo počtu mincí vracet konkrétní řešení.
- Vracet všechna minimální řešení:
 - např. pro mince 1, 2, 3, 4 a částku 5 existují řešení [2, 3] a [1, 4].

Backtracking

- Postupně, typicky rekurzivně, se buduje řešení.
- Lokální volba:
 - Posoudit, jaké jsou možnosti udělat *jeden krok*.
 - *Postupně zkoušet*.
- Včasné rozpoznání, že *tato cesta už nikam nevede*,
 - tzv. „odříznutí“ (pruning) větve rekurze.
- Chceme *jedno řešení*: návrat skrz všechny úrovně.
- Chceme *všechna řešení*: předávání datové struktury s řešeními.
- Chceme *nejlepší řešení*:
 - předáváme si informaci o aktuálně nejlepší hodnotě,
 - nemá smysl pokračovat cestami, které hodnotu nezlepší.